



Section 6 / 24

Assembly File Structure

As we learn various Assembly instructions in the coming sections, we'll constantly be writing code, assembling it, and debugging it. This is the best way to learn what each instruction does. So, we need to learn the basic structure of an Assembly code file and then assemble it and debug it.

In this section, we'll go through the basic structure of an Assembly file, and in the following two sections, we will cover assembling it and debugging it. We will be working on a template `Hello World!` Assembly code as a sample, to first learn the general structure of an assembly file and then how to assemble it and debug it. Let us start by taking a look at and dissecting a sample `Hello World!` Assembly code template:

```
global _start

section .data
message: db "Hello HTB Academy!"

section .text
_start:
    mov     rax, 1
```

nasm

```

mov     rdi, 1
mov     rsi, message
mov     rdx, 18
syscall

mov     rax, 60
mov     rdi, 0
syscall

```

This Assembly code (once assembled and linked) should print the string ' `Hello HTB Academy!` ' to the screen. We won't go into detail on how this is processed just yet, but we need to understand the main elements of the code template.

Assembly File Structure

First, let's examine the way the code is distributed:

	Labels	Instruc tions	Operands
Directives		global	<code>_start</code>
Sections	<code>message:</code>	section db	.data "Hello HTB Academy!"
	<code>_start:</code>	section	.text
		mov mov mov mov syscall mov mov syscall	rax, 1 rdi, 1 rsi, message rdx, 18 rax, 60 rdi, 0

Looking at the vertical parts of the code, each line can have three elements:

1. Labels

2. Instructions

3. Operands

We have discussed the `instructions` and their `operands` in the previous sections, and we'll go into detail on various assembly instructions in the coming sections. In addition to that, we can define a `label` at each line. Each label can be referred to by `instructions` or by `directives`.

Next, if we look at the code line-by-line, we see that it has three main parts:

Section	Description
<code>global _start</code>	This is a <code>directive</code> that directs the code to start executing at the <code>_start</code> label defined below.
<code>section .data</code>	This is the <code>data</code> section, which should contain all of the variables.
<code>section .text</code>	This is the <code>text</code> section containing all of the code to be executed.

Both the `.data` and `.text` sections refer to the `data` and `text` memory segments, in which these instructions will be stored.

Directives

An Assembly code is line-based, which means that the file is processed line-by-line, executing the instruction of each line. We see at the first line a directive `global _start`, which instructs the machine to start processing the instructions after the `_start` label. So, the machine goes to the `_start` label and starts executing the instructions there, which will print the message on the screen. This will be covered more thoroughly in the `Control Instructions` sections.

Variables

Next, we have the `.data` section. The `data` section holds our variables to make it easier for us to define variables and reuse them without writing them multiple times. Once we run our program, all of our variables will be loaded into memory in the `data` segment.

When we run the program, it will load any variables we have defined into memory so that they will be ready for usage when we call them. We will notice later in the module that by the time we start executing instructions at the `_start` label, all of our variables will be already loaded into memory.

We can define variables using `db` for a list of bytes, `dw` for a list of words, `dd` for a list of digits, and so on. We can also label any of our variables so we can call it or reference it later. The following are some examples of defining variables:

Instruction	Description
<code>db 0x0a</code>	Defines the byte <code>0x0a</code> , which is a new line.
<code>message db 0x41, 0x42, 0x43, 0x0a</code>	Defines the label <code>message => ABC\n</code> .

Instruction	Description
<code>message db "Hello World!", 0x0a</code>	Defines the label <code>message => Hello World!\n</code> .

Furthermore, we can use the `equ` instruction with the `$` token to evaluate an expression, like the length of a defined variable's string. However, the labels defined with the `equ` instruction are constants, and they cannot be changed later.

For example, the following code defines a variable and then defines a `constant` for its length:

```
nasm
section .data
    message db "Hello World!", 0x0a
    length equ $-message
```

Note: the `$` token indicates the `current distance` from the `beginning of the current section`. As the `message` variable is at the beginning of the `data` section, the current location, i.e., value of `$` , equals the length of the string. For the scope of this module, we will only use this token to calculate lengths of strings, using the same line of code shown above.

Code

The second (and most important) section is the `.text` section. This section holds all of the assembly instructions and loads them to the `text` memory segment. Once all instructions are loaded into the `text` segment, the processor starts executing them one after another.

The default convention is to have the `_start` label at the beginning of the `.text` section, which -as per the `global _start` directive- starts the main code that will be executed as the program runs. As we will see later in the module, we can define other labels within the `.text` section, for loops and other functions.

The `text` segment within the memory is read-only, so we cannot write any variables within it. The `data` section, on the other hand, is read/write, which is why we write our variables to it. However, the `data` segment within the memory is not executable, so any code we write to it cannot be executed. This separation is part of memory protections to mitigate things like buffer overflows and other types of binary exploitation.

Tip: We can add comments to our assembly code with a semi-colon `;`. We can use comments to explain the purpose of each part of the code, and what each line is doing. Doing so will save us a lot of time in the future if we ever revisit the code and need to understand it.

With this, we should understand the basic structure of an Assembly file.

[< Previous](#)

Section 6 / 24

 +20

[Next](#)[Mark Complete & Next](#)